

1) **fr** — fração do espaço disponível

O que é: unidade que divide o espaço *restante* do container em fatias (fractions).

Exemplo: `grid-template-columns: 1fr 2fr;` → a segunda coluna recebe o dobro do espaço restante da primeira.

```
.container { display: grid; grid-template-columns: 1fr 2fr; gap: 10px; }
```

Visual esperado:

- Se sobrar 600px: primeira = 200px, segunda = 400px.

Desafio rápido:

Crie 3 colunas: `1fr 1fr 2fr`. Insira 3 caixas e veja qual ocupa mais espaço.

2) **repeat()** — repetir padrões

O que é: atalho para repetir colunas/linhas sem escrever tudo manualmente.

Exemplo: `repeat(4, 1fr)` → 4 colunas iguais de `1fr` cada.

```
.grid { grid-template-columns: repeat(3, 1fr); }
```

Desafio:

Troque `repeat(3, 1fr)` por `repeat(2, 2fr)` e observe a diferença.

3) **minmax(min, max)** — limite mínimo/máximo para uma faixa

O que é: define **mínimo** e **máximo** para uma coluna/linha. Útil para responsividade.

Exemplo: `minmax(150px, 1fr)` significa:

- **Nunca menor que 150px**, e
- **Até 1fr** se houver espaço.

```
.grid { grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); }
```

Casos práticos:

- Em telas pequenas, cada coluna **não fica menor que 150px** (vai para nova linha).
- Em telas maiores, as colunas crescem e dividem o espaço (`1fr`).

Desafio:

Mude `minmax(150px, 1fr)` para `minmax(100px, 300px)` e veja como as colunas param de crescer

depois de 300px.

4) **auto-fit** vs **auto-fill** — ajustando quantas colunas cabem

auto-fill

- Tenta **encher** a linha com quantas colunas couberem — mesmo que algumas fiquem vazias.
- Se houver espaço para 5 colunas, ele cria 5 "slots". Quando ficar estreito, os slots "vazios" ocupam espaço invisível.

auto-fit

- **Agrupar colunas vazias**, fazendo com que as colunas reais *se expandam* para ocupar o espaço disponível.
- Se você tiver menos itens que slots, **auto-fit** faz os itens ocuparem o espaço sobrando; **auto-fill** mantém os slots vazios.

Exemplo visual com **minmax**:

```
/* auto-fill */
.grid { grid-template-columns: repeat(auto-fill, minmax(150px, 1fr)); }

/* auto-fit */
.grid { grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); }
```

Quando usar:

- Use **auto-fit** quando quiser que os itens **se ajustem e expandam** ocupando espaço disponível.
- Use **auto-fill** quando quiser **manter "slots" fixos** (menos comum para UIs responsivas).

Desafio:

Crie 3 cartões dentro de um container com `repeat(auto-fill, minmax(150px, 1fr))` e depois troque para **auto-fit**. Reduza a largura da janela e compare o comportamento.

5) **gap** — espaçamento entre células (colunas e linhas)

O que é: substitui margin entre itens do grid sem afetar o fluxo.

```
.grid { gap: 20px; } /* mesma distância horizontal e vertical */
.grid { gap: 10px 20px; } /* row-gap column-gap */
```

Desafio:

Remova **gap** e tente usar **margin** nos itens — observe desalinhamentos indesejados.

6) `grid-template-areas` e `grid-area` — layout semântico

O que é: nomear áreas do grid para posicionar itens facilmente.

```
.container {
  grid-template-areas:
    "header header"
    "nav    main"
    "nav    aside"
    "footer footer";
  grid-template-columns: 200px 1fr;
}
header { grid-area: header; }
nav     { grid-area: nav;   }
main    { grid-area: main;  }
aside   { grid-area: aside; }
footer  { grid-area: footer; }
```

Vantagem: mais legível e fácil de reorganizar com media queries.

Desafio:

Troque as posições de `nav` e `aside` apenas alterando `grid-template-areas`.

7) `grid-column` / `grid-row` — posicionamento por índices

Sintaxe curta: `grid-column: start / end;` onde `start` e `end` são linhas de grade.

```
.item { grid-column: 1 / 3; /* ocupa colunas 1 e 2 */ }
```

Atalhos:

- `grid-column: span 2;` → ocupa 2 colunas a partir da posição atual.
- Também existe `grid-row: 2 / span 2;`

Desafio:

Crie 6 itens: deixe o primeiro ocupar `grid-column: 1 / 4;` (3 colunas).

8) Grid explícito vs implícito

- **Explícito:** você definiu `grid-template-columns/rows` — essas são as linhas/colunas planejadas.
- **Implícito:** quando adiciona mais itens que o grid explicitamente definido, o browser cria linhas/colunas **implícitas** (padrão `auto`). Controle com `grid-auto-rows` / `grid-auto-columns`.

```
.grid { grid-auto-rows: 100px; } /* as linhas criadas automaticamente terão 100px */
```

Desafio:

Defina `grid-template-rows: 100px 100px;` e adicione 6 itens. Observe as linhas implícitas criadas.

9) Alinhamentos: `justify-items`, `align-items`, `place-items`, `justify-content`, `align-content`

- `justify-items` — alinha conteúdo dentro da **célula** horizontalmente (start / center / end / stretch).
- `align-items` — alinha conteúdo dentro da **célula** verticalmente.
- `place-items` — atalho para os dois: `place-items: center stretch;`
- `justify-content` — alinha **todo o grid** horizontalmente dentro do container.
- `align-content` — alinha **todo o grid** verticalmente dentro do container.

Exemplo:

```
.grid { justify-items: center; align-items: start; }
```

Desafio:

Em um grid de 3x3, centralize os conteúdos das células: `place-items: center;`

10) Boas práticas rápidas (resumo)

- Prefira `gap` em vez de `margin` entre itens.
 - Use `grid-template-areas` para layouts complexos — fica mais legível.
 - Para galerias/responsividade, combine `repeat(auto-fit, minmax(...))`.
 - Teste `minmax()` com valores absolutos e relativos (px, rem, %, fr).
 - Evite definir widths fixas demais se quiser responsividade.
-

11) Mini-atividade final para a sala (passo a passo rápido)

Objetivo: criar uma galeria responsiva de cartões que muda de 4 → 2 → 1 coluna conforme a largura.

HTML:

```
<div class="gallery">
  <div class="card">1</div>
  <div class="card">2</div>
  <div class="card">3</div>
  <div class="card">4</div>
```

```
<div class="card">5</div>
</div>
```

CSS sugerido:

```
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
  padding: 16px;
}
.card { padding: 24px; border: 1px solid #ddd; text-align: center; }
```

O que testar (passos para alunos):

1. Reduzir a janela — quantas colunas aparecem?
2. Mudar 180px para 120px — o que muda?
3. Trocar auto-fit por auto-fill — notar a diferença visual.